

Home Assignment:

1. Add the resume folder containing the resume files to the existing workspace.
2. write a prompt to implement a batch ingestion workflow for resumes.
3. Configure the workflow to process resumes in batches of 5 or 10 files per run.
4. Validate the ingestion process by testing multiple batches and verifying that all resumes are successfully ingested.

Prompt

ICEPOT

Instructions

Analyse the current application as Act as a Solution Architect and RAG Engineer

Help me to implement a batch ingestion workflow for 199 resumes in the path "C:\Gen AI Course\Class\Week9_Agent_Building_Best Practice\Sunday\Ingestion Pipeline\Resumes\Resumes"

Configure the workflow to process resumes in batches of 5 files per run.

the resumes should be ingested in MongoDB Database & Collections given in the .env using the MISTRAL_API_KEY

Validate the ingestion process by testing multiple batches and verifying that all resumes are successfully ingested

do not change the existing functionalities [STRICTLY]

I am a beginner, so guide me step by step.

If any important requirement is missing, ask me a question instead of assuming it.

Context / Background

You are helping me build a Resume RAG application.

The main flow is:

```
```text
```

Resume PDF



Upload



Extract + Clean Text



Parse Resume + Skills + Metadata



Generate Embedding



Store in MongoDB



User Query



Query Preprocessing



Hybrid Search



Reranking + Deduplication



Relevant Resume Context



Prompt + Query + Context



LLM



Final Result

**\*\*Initial Technology Preference\*\***

Use these as starting preferences. If something needs to change, explain it and ask me first.

Backend: Node.js + TypeScript + Express

Frontend: React

Database / Vector Store: MongoDB

Embedding Model: Mistral AI

LLM: OpenAI / Groq / Anthropic

**\*\*Injection / Guardrails\*\***

Do not invent resume information.

Do not invent requirements.

Do not expose API keys or secrets.

Use only the information available in the resume when extracting candidate details.

If information is missing, mark it as missing or ask me.

Keep ingestion and retrieval as separate modules but within the same application.

Keep the design simple and suitable for a beginner.

Avoid unnecessary complexity.

**\*\*Persona:\*\***

Act as a Solution Architect and RAG Engineer.

Also act as a beginner-friendly mentor.

Keep technical explanations simple and clear.

**\*\*Tone:\*\***

Professional, simple, clear, and concise.

Questions Asked and Answer:

Q: Folder has 20+ .docx/.doc files, but pipeline only accepts PDF. How to handle them?

**A: Skip non-PDF files - Skip .docx/.doc, log them in the report. No new parsing code.**

Q: Some resumes repeat with '(1)' suffixes (likely duplicate copies). How to handle duplicates?

**A: Skip exact duplicate files - Pre-check content hash and skip exact duplicate files before calling embedding API.**

Q: Should batch ingestion be a standalone script calling the service directly, or an HTTP client hitting the running server?

**A: Standalone script (direct service call) - No server needed; calls ResumeIngestionService directly. Safest**

The screenshot shows a Visual Studio Code editor with a file explorer on the left, a code editor in the center, and a chat window on the right. The code editor displays a script for resume ingestion, and the chat window shows a summary of the batch ingestion workflow.

**Code Editor Content:**

```
01_Resume_Ingestion_Phasewise_Implementation 1.md
312 # PHASE 3 - Ingestion Module Folder Setup
366 ## Phase gate
371
372 # PHASE 4 - Secure PDF Upload
373
374 ## Goal
375
376 Accept PDF resumes safely.
377
378 ## Install
379
380 ```bash
381 npm install multer pdf-parse
382 npm install -D @types/multer
383 ```
384
385 ## File
386
387 ```text
388 src/config/multerConfig.ts
```

**Chat Window Content:**

Batch ingestion workflow for resumes

164 Resumes are ingested in the Mon... 2/3

Batch ingestion completed successfully.  
Final summary:

Metric	Count
Total files found in source folder	199
Unique PDFs ingested	164
Succeeded	164
Failed	0
Skipped (non-PDF: .docx/.doc)	19
Skipped (exact duplicates)	16

All 33 batches (5 files each, last batch had 4) ran with a 3s delay between batches, no failures, no rate-limit errors from Mistral. Full per-file details (fileName, resumeld, timings) are saved at:

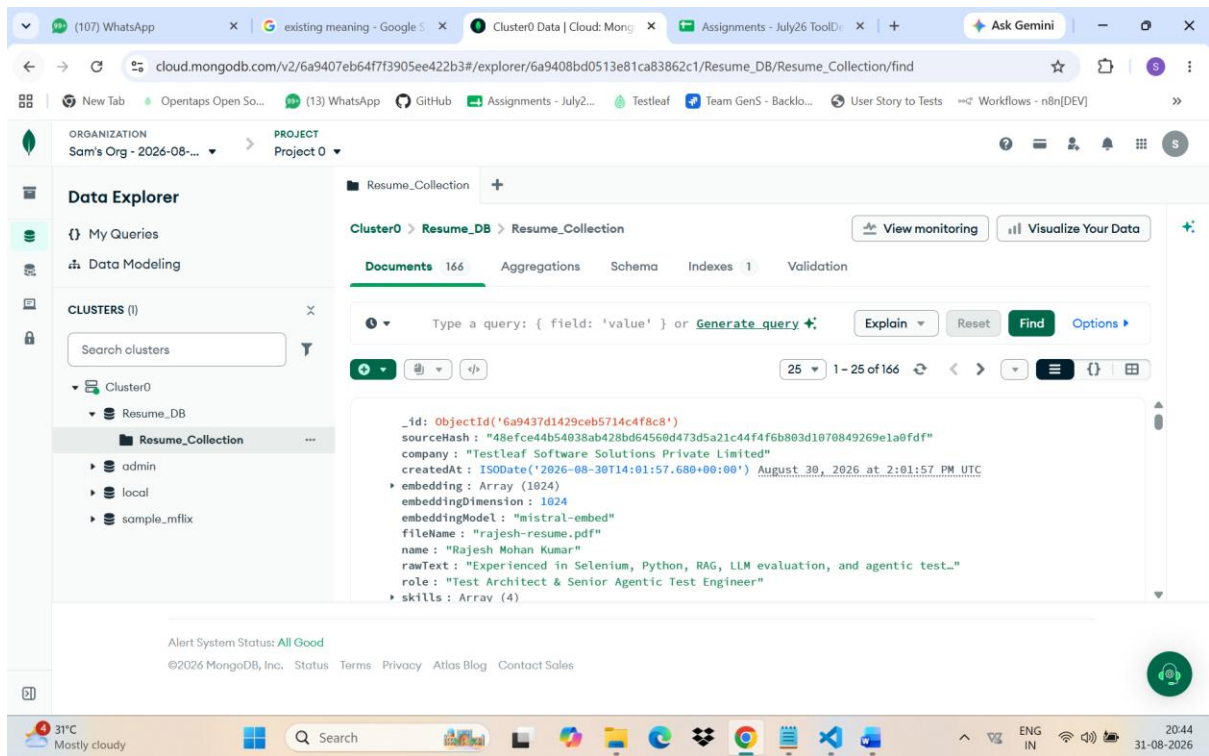
Implement batch ingestion wor...

01\_Resume\_Ingestion\_Phasewise\_Impleme...

Describe what to build

Agent Auto

Local Allow all



Batch ingestion completed successfully. Final summary:

Metric	Count
Total files found in source folder	199
Unique PDFs ingested	<b>164</b>
Succeeded	<b>164</b>
Failed	0
Skipped (non-PDF: .docx/.doc)	19
Skipped (exact duplicates)	16

All 33 batches (5 files each, last batch had 4) ran with a 3s delay between batches, no failures, no rate-limit errors from Mistral. Full per-file details (fileName, resumeld, timings) are saved